

Segurança em Redes Corporativas: Análise, Implementação e Avaliação de Mecanismos de Proteção

Evelyn Caroline Morais Targino – 221006404
Lucas Vasconcelos Pirani Carneiro – 251031785
Leonardo Marques de Paula Junior – 221029015
Vinícius Geraldo dos Santos Guerra – 190096454

Abstract—This report analyzes security mechanisms and common attacks in corporate networks, focusing on HTTP/HTTPS, TLS, digital certificates, firewalls and vulnerability assessment. We present an experimental setup using packet captures and tools (Wireshark, OpenSSL, nmap) to identify insecure services and assess risks. Mitigation strategies are proposed, including PKI adoption, firewall policies and secure configuration practices. Results highlight the need for layered defenses and proactive incident response.

Index Terms—Segurança Computacional; Network Security; TLS; HTTPS; Vulnerability Analysis; Firewalls

I. INTRODUÇÃO

A segurança em redes corporativas é essencial para preservar disponibilidade, confidencialidade e integridade dos ativos da organização. Este projeto aborda protocolos (HTTP/HTTPS, TLS), certificados digitais, controle de acesso e análise de vulnerabilidades em um ambiente experimental. O objetivo desse relatório é identificar riscos via capturas e varreduras, avaliar impactos e propor contramedidas práticas. O relatório apresenta fundamentação teórica, descrição do ambiente, resultados experimentais, estudo de caso e conclusões.

II. FUNDAMENTAÇÃO TEÓRICA

A. Protocolos HTTP e HTTPS

O HyperText Transfer Protocol (HTTP) é o protocolo da camada de aplicação usado na Web, que define como clientes (por exemplo, navegadores) solicitam recursos a servidores e como os servidores transferem esses recursos de volta ao cliente [1]. HTTP segue um modelo cliente-servidor: quando um usuário solicita uma página (por exemplo, clicando em um link), o cliente envia uma mensagem de requisição ao servidor, que responde com uma mensagem contendo os objetos requisitados [1]. Como protocolo de transporte subjacente, o HTTP utiliza o TCP; antes de trocar mensagens, o cliente estabelece uma conexão TCP com o servidor [1]. HTTP é um protocolo sem estado (stateless): o servidor não mantém informações sobre requisições anteriores de um cliente, de modo que requisições repetidas por um mesmo objeto são tratadas de forma independente [1].

Mensagens HTTP são textos em ASCII estruturados em requisições e respostas [1].

- **Requisição HTTP:** A mensagem inicia com uma linha de requisição contendo o método (por exemplo, GET, POST, HEAD, PUT, DELETE), a URL (ou campo da URL) e a versão do protocolo; seguem linhas de cabeçalho que fornecem metadados (por exemplo, Host, User-Agent, Accept-Language); e, opcionalmente, um corpo de entidade usado para enviar dados (por exemplo, dados de formulários em POST) [1].
- **Resposta HTTP:** A mensagem inicia com uma linha de status que inclui a versão do protocolo, um código de status (por exemplo, 200 OK, 404 Not Found) e uma frase de status; seguem linhas de cabeçalho com metadados (por exemplo, Date, Server, Last-Modified, Content-Type); e um corpo de entidade que contém o recurso solicitado (por exemplo, HTML, imagem) [1].

O HTTP apresenta limitações de segurança importantes, pois foi originalmente projetado supondo confiança mútua entre pontos finais, o que é inadequado na Internet atual [1]. Entre as principais fragilidades estão:

- **Transmissão em texto claro:** O HTTP transmite dados sem criptografia, logo informações sensíveis (senhas, números de cartão, etc.) podem ser interceptadas por sniffers em nós intermediários [1].
- **Falta de integridade:** Não há garantia intrínseca de que a mensagem não foi modificada durante o trânsito [1].
- **Falta de autenticação:** O cliente não possui meios confiáveis embutidos no protocolo para verificar se está se comunicando com o servidor legítimo, possibilitando ataques de spoofing e phishing [1].

O HTTPS corresponde ao HTTP operando sobre o Transport Layer Security (TLS), que acrescenta serviços de segurança ao transporte de dados [1]. O uso de TLS busca resolver as limitações do HTTP ao prover:

- **Confidencialidade (criptografia):** Protege o conteúdo contra interceptação, garantindo que apenas remetente e destinatário possam ler os dados [1].
- **Integridade:** Permite detectar alterações ou corrupções nos dados em trânsito [1].
- **Autenticação do servidor:** Por meio de certificados digitais emitidos por Autoridades Certificadoras (CAs), o

cliente pode verificar que está se conectando ao servidor correto, reduzindo riscos de spoofing [1].

Além disso, a implantação típica de HTTPS inclui um handshake TLS que negocia algoritmos criptográficos e estabelece chaves de sessão seguras antes da transferência dos dados HTTP, proporcionando proteção contínua durante a conexão [1].

B. TLS (Transport Layer Security)

O Transport Layer Security (TLS), sucessor do SSL, é um protocolo que fornece serviços de segurança para conexões TCP; do ponto de vista do desenvolvedor funciona como um aprimoramento do transporte que adiciona confidencialidade, integridade e autenticação às comunicações [1]. Seus objetivos principais incluem garantir confidencialidade (apenas remetente e destinatário lêem os dados), integridade dos dados (detecção de alterações em trânsito), autenticação do servidor (verificar a identidade do servidor) e, opcionalmente, autenticação do cliente (verificar a identidade do cliente) [1].

O handshake TLS é a fase inicial em que as partes negociam parâmetros de segurança e estabelecem segredos compartilhados [1]. Etapas principais do handshake:

- **Estabelecimento de conexão TCP:** Antes do handshake TLS ocorre o three-way TCP handshake para abrir a conexão de transporte [1].
- **ClientHello:** O cliente envia uma mensagem ClientHello contendo um nonce (número aleatório), a versão TLS suportada e uma lista de conjuntos de cifrões e algoritmos que suporta [1].
- **ServerHello e certificado:** O servidor responde com ServerHello (seu nonce e a escolha dos algoritmos) e envia seu certificado digital que contém a chave pública do servidor [1].
- **Pre-Master Secret (PMS):** O cliente gera um Pre-Master Secret, o cifra com a chave pública do servidor (ou usa um método de troca de chaves como Diffie-Hellman) e o envia ao servidor [1].
- **Derivação de chaves:** A partir do PMS e dos nonces, ambas as partes derivam um Master Secret e, em seguida, as chaves de sessão simétricas e chaves de integridade (MAC/HMAC) para uso nas mensagens da aplicação [1].
- **Verificação final:** As partes trocam mensagens de Finished que contêm hashes de todo o handshake para verificar que o processo não foi alterado (proteção contra ataques man-in-the-middle) [1].

O TLS combina criptografia assimétrica e simétrica para equilibrar segurança e desempenho [1].

- **Criptografia assimétrica (chave pública):** Usada principalmente no handshake para autenticar o servidor (via certificado) e proteger a troca inicial de segredos (por exemplo, cifrando o Pre-Master Secret ou usando autenticação na troca Diffie-Hellman) [1].
- **Criptografia simétrica:** Após o handshake, os dados da aplicação são criptografados com chaves simétricas de sessão derivadas do Master Secret, pois criptografia

simétrica é muito mais eficiente para transferir grandes volumes de dados [1].

O TLS assegura confidencialidade, integridade e autenticidade por meio de mecanismos combinados [1].

- **Confidencialidade:** A criptografia simétrica das mensagens de aplicação impede que terceiros leiam o conteúdo transmitido entre cliente e servidor [1].
- **Integridade:** Cada registro TLS inclui um código de autenticação (por exemplo, HMAC) calculado sobre os dados e um número de sequência; isso permite detectar alterações e também evita reordenação ou repetição de registros [1].
- **Autenticidade:** A verificação do certificado do servidor, feita com a chave pública da Autoridade Certificadora (CA), autentica a identidade do servidor; opcionalmente, o servidor pode exigir certificados de cliente para autenticação mútua [1].

Além desses pontos, é importante notar que a segurança prática do TLS depende de configurações corretas: versões e conjuntos de cifrões desatualizados ou certificados mal emitidos/gerenciados podem introduzir vulnerabilidades, por isso recomenda-se usar configurações modernas (por exemplo, TLS 1.2+ com conjuntos de cifrões robustos) e gerenciar corretamente certificados e chaves [1].

C. Certificados Digitais e PKI

Um certificado digital é um documento eletrônico que vincula uma chave pública a uma entidade específica (por exemplo, uma pessoa, um servidor ou um roteador) e contém a chave pública do proprietário junto com informações de identificação globais e metadados relevantes [1]. A finalidade principal do certificado é atestar que a chave pública pertence realmente à entidade declarada, impedindo que um atacante apresente sua própria chave como se fosse de outra pessoa (por exemplo, para interceptar comunicações ou forjar assinaturas) [1].

As Autoridades Certificadoras (CAs) são entidades de confiança responsáveis por verificar identidades e emitir certificados digitais [1]. As funções essenciais das CAs incluem:

- **Verificação de identidade:** A CA realiza procedimentos de validação para confirmar que a entidade solicitante é realmente quem afirma ser antes de emitir um certificado [1].
- **Emissão e assinatura:** Após a verificação, a CA cria o certificado e o assina digitalmente usando sua chave privada, vinculando a identidade do solicitante à sua chave pública e permitindo que terceiros verifiquem essa associação [1].

A cadeia de confiança dos certificados digitais baseia-se na confiança em uma CA raiz ou em uma cadeia de CAs intermediárias [1]. O funcionamento típico é:

- **Confiança na CA:** O usuário (por exemplo, Alice) confia na(s) CA(s) raiz que estão pré-instaladas em seu sistema ou navegador e, portanto, confia implicitamente nas assinaturas por elas emitidas [1].

- **Verificação da assinatura:** Quando Alice recebe o certificado de Bob, ela usa a chave pública da CA (ou da CA intermediária apropriada) para verificar a assinatura digital presente no certificado [1].
- **Validação da associação:** Se a assinatura for válida e o certificado estiver dentro do período de validade e não estiver revogado, Alice aceita que a chave pública contida no certificado pertence a Bob, conforme atestado pela CA [1].

Uma Infraestrutura de Chaves Públicas (PKI) é o sistema abrangente de hardware, software, pessoas, políticas e procedimentos necessários para criar, gerenciar, distribuir, usar, armazenar e revogar certificados digitais e chaves públicas [1]. A operação de uma PKI envolve:

- **Padrões e formatos:** Uso de normas como X.509 para o formato dos certificados e RFCs relevantes para procedimentos de gerenciamento e validação [1].
- **Gestão de ciclo de vida:** Emissão, renovação, publicação (por exemplo, em repositórios ou através de OCSP/CRL), e revogação de certificados conforme políticas definidas [1].
- **Componentes organizacionais:** CAs (raiz e intermediárias), Autoridades de Registro (RAs) que validam solicitações, repositórios de certificados e mecanismos de verificação de revogação (CRL, OCSP) [1].

Existem modelos alternativos ao modelo centralizado de CAs; por exemplo, o PGP usa uma “web of trust”, em que usuários assinam chaves uns dos outros sem depender exclusivamente de uma autoridade central [1]. Em todos os casos, a segurança prática depende de políticas, procedimentos e gestão cuidadosa de chaves e certificados, pois erros de emissão, configuração ou revogação podem comprometer a confiança do sistema [1].

D. Firewall e Controle de Acesso

Um firewall é uma combinação de hardware e software que isola a rede interna de uma organização da Internet pública, controlando o tráfego que entra e sai e permitindo a passagem apenas dos pacotes autorizados pela política de segurança local [1]. Sua função principal é gerenciar o fluxo de tráfego entre redes (por exemplo, Internet e rede interna) para proteger recursos internos contra acessos indevidos e ataques [1]. Para ser eficaz, um firewall deve garantir que todo o tráfego passe por ele, que apenas o tráfego autorizado seja permitido e que o próprio dispositivo seja resistente a penetrações [1].

A filtragem de pacotes examina informações do cabeçalho de cada datagrama e aplica regras para decidir aceitar ou descartar o pacote [1]. Existem duas abordagens principais:

- **Filtros de pacotes tradicionais (stateless):** Analisam cada datagrama de forma independente, consultando campos do cabeçalho como endereços IP de origem/destino, protocolo (TCP, UDP, ICMP), portas de origem/destino e flags TCP (por exemplo, SYN, ACK) para tomar decisões de aceitação ou bloqueio [1].
- **Filtros com estado (stateful):** Mantêm uma tabela de conexões ativas (estado), permitindo ao firewall reconhecer

se um pacote de entrada pertence a uma conexão iniciada internamente e oferecer verificação adicional contra pacotes fora de contexto ou tentativas de mapeamento de rede [1].

O controle de acesso baseado em regras (ACLs) contribui para a segurança ao permitir que administradores definam políticas finas sobre quais fluxos são permitidos ou negados [1]. Exemplos de benefícios práticos incluem:

- **Bloqueio de conexões indesejadas:** Permitir apenas serviços públicos explícitos (por exemplo, servidor web na porta 80) e negar o restante do tráfego de entrada [1].
- **Mitigação de ataques:** prevenir vetores de ataque simples, como certas mensagens ICMP ou tráfego para endereços de broadcast, que podem ser usados em ataques de reflexão ou mapeamento de rede [1].
- **Controle de recursos:** Restringir aplicações que consomem muita largura de banda (por exemplo, fluxos UDP de mídia) para preservar recursos internos [1].
- **Proteção da topologia:** Impedir respostas que auxiliem no mapeamento da rede (por exemplo, certos tipos de ICMP), dificultando reconhecimento por invasores [1].

Ferramentas comuns em sistemas Linux para implementar regras de firewall incluem iptables (e seus sucessores, como nftables) e interfaces simplificadas como o UFW (Uncomplicated Firewall) [1]. Características e funcionamento prático:

- **iptables:** Software tradicional de filtragem de pacotes para Linux que permite criar regras detalhadas aplicadas a cada pacote que atravessa as cadeias de processamento; as regras são avaliadas de cima para baixo até ocorrer correspondência, o que determina permitir (ACCEPT) ou negar (DROP/REJECT) o pacote [1].
- **Stateful vs. stateless:** iptables suporta inspeção de estado (conntrack), possibilitando regras que dependem do estado da conexão (por exemplo, permitir pacotes ESTABLISHED,RELATED) [1].
- **UFW:** É uma interface de alto nível para facilitar a administração de regras de firewall (normalmente sobre iptables/nftables), oferecendo comandos mais simples para tarefas comuns; porém, detalhes e limitações do UFW não são aprofundados nas fontes citadas e devem ser consultados separadamente quando necessário [1].
- **Aplicação de ACLs:** Tanto com iptables quanto com UFW, o administrador define políticas (listas de regras) que são avaliadas sequencialmente; a ordem das regras é importante e uma regra genérica no topo pode bloquear correspondências posteriores desejadas [1].

Em resumo, firewalls e controle de acesso baseado em regras são componentes fundamentais para a defesa de redes, combinando filtragem de pacotes (stateless) e inspeção de estado (stateful) com políticas definidas pelo administrador; ferramentas como iptables e UFW permitem implementar essas políticas na prática, desde regras granulares até configurações simplificadas para administradores [1].

E. Análise de Vulnerabilidades

Uma vulnerabilidade é uma falha de projeto ou de implementação em software ou sistemas que pode ser explorada por agentes maliciosos para comprometer a segurança, violar a privacidade ou tornar serviços inoperantes [1]. Entre classes comuns estão a corrupção de memória (erros que permitem leitura/escrita fora dos limites) e vulnerabilidades de dia zero (0-day), que são exploradas antes de serem conhecidas ou corrigidas [1].

A enumeração de serviços é o processo de identificar quais aplicações estão “ouvindo” em portas específicas de um host, tipicamente realizado com port scanners como o nmap, que verificam o estado de portas TCP/UDP (aberta, fechada, filtrada) [1]. Essa etapa é importante porque permite mapear quais serviços estão expostos e associá-los a versões ou implementações conhecidas, facilitando a identificação de falhas exploráveis (por exemplo, um servidor SQL com vulnerabilidade de buffer overflow) [1].

O mapeamento de riscos em uma infraestrutura de rede envolve reunir informações sobre topologia, serviços e ativos para identificar e priorizar pontos fracos [1]. Atividades típicas incluem:

- **Reconhecimento e escaneamento:** Uso de port scanning (nmap) e traceroute para descobrir serviços ativos, portas e roteadores intermediários [1].
- **Análise de vetores de ataque:** Avaliar suscetibilidade a sniffing (especialmente em redes sem fio), IP spoofing e outras técnicas de evasão [1].
- **Verificação de credenciais:** Procurar vazamento de chaves de API, senhas ou chaves privadas em repositórios públicos e em imagens de software [1].
- **Auditoria de software/hardware:** Confirmar se componentes passaram por testes, validação de integridade e estão configurados de forma segura antes da implantação [3].

Para mitigar vulnerabilidades, adota-se uma estratégia multicamadas (defesa em profundidade) combinando práticas de desenvolvimento, operações e proteção perimetral [3]. Medidas recomendadas incluem:

- **Desenvolvimento seguro:** Usar linguagens e técnicas que reduzam erros (por exemplo, migrar código crítico para linguagens com gerenciamento de memória seguro como Rust), aplicar análise estática/ dinâmica e revisão de código automatizada [3].
- **Gestão de atualizações:** Manter software e dependências atualizados para corrigir falhas conhecidas e reduzir a superfície de ataque [3].
- **Defesas de perímetro:** Empregar firewalls, filtragem de pacotes/stateful e gateways de aplicação para isolar serviços e bloquear tráfego malicioso [3].
- **Criptografia e autenticação:** Usar TLS/HTTPS e certificados digitais para proteger confidencialidade, integridade e autenticidade das comunicações [3].
- **Sistemas de monitoramento (IDS/IPS):** Implantar sensores e ferramentas de inspeção profunda de pacotes

(DPI) e detecção de anomalias para identificar e responder a ataques em tempo real [3].

- **Validação de integridade:** Aplicar hashes criptográficos e assinaturas digitais para garantir a integridade do software na cadeia de suprimentos [3].
- **Configuração segura e CI/CD:** Adotar políticas de “secure by default”, automação de testes (unitários, integração, sistema) em pipelines CI/CD e políticas de revisão antes da entrega para evitar erros de configuração e regressões [3].

Em resumo, a análise de vulnerabilidades combina descoberta técnica (enumeração de serviços, escaneamento) com avaliação de risco e aplicação de controles técnicos e processuais; a eficácia depende tanto de controles tecnológicos quanto de boas práticas organizacionais (gestão de patches, treinamento e políticas) [3].

F. Ataques em Redes Corporativas

Um ataque em rede corporativa consiste em ações realizadas por agentes maliciosos com o objetivo de danificar sistemas, violar a privacidade ou tornar serviços e recursos indisponíveis em instituições como empresas, universidades e órgãos governamentais [1]. Esses ataques exploram tanto vulnerabilidades técnicas quanto falhas humanas, incluindo a distribuição de malware (vírus, worms, spyware), exploração de falhas de implementação (por exemplo, corrupção de memória) e técnicas de engenharia social para roubo de credenciais [1], [3].

A seguir será explicado ataques clássicos e comuns em redes corporativas.

Ataque de phishing: O *phishing* é uma técnica de ataque ao nível de aplicação que utiliza comunicações enganosas (tipicamente e-mail) para induzir usuários a revelar credenciais ou executar ações inseguras [3]. Quando um funcionário é vítima de *phishing*, o atacante pode obter acesso a credenciais de serviços internos (VPN, sistemas corporativos) e, a partir daí, mover-se lateralmente pela rede para comprometer ativos críticos [3]. Casos reais demonstram que ataques que começam com roubo de credenciais podem levar a violações de grande impacto, como o incidente envolvendo a RSA e subsequentes invasões a empresas parceiras [3].

Ataques DDoS: Um ataque de Negação de Serviço Distribuída (DDoS) utiliza um conjunto de máquinas comprometidas (botnet) para inundar um alvo com tráfego simultâneo ou requisições, com o objetivo de esgotar recursos de rede ou do host [1]. Os vetores incluem inundações de largura de banda (saturação do link), inundações de conexão (esgotamento de tabelas de estado/recursos do servidor) e exploração de vulnerabilidades específicas para travar serviços [1]. O impacto típico é indisponibilidade de serviços para usuários legítimos, com implicações financeiras e de reputação; grandes incidentes já afetaram provedores e plataformas de grande escala por horas ou dias [1].

Ransomware: *Ransomware* é uma classe de malware que cifra dados ou bloqueia sistemas, exigindo pagamento (resgate) para restaurar o acesso [3]. Frequentemente o ransomware

se propaga internamente após o atacante obter acesso inicial via credenciais comprometidas ou explorando vulnerabilidades não corrigidas. Características comuns incluem criptografia de arquivos, exfiltração de dados (para chantagear a vítima) e movimentos laterais para maximizar impacto operacional, podendo forçar paralisação de serviços críticos [2].

Spoofing: *Spoofing* refere-se à falsificação de identificadores de rede (por exemplo, IP spoofing), onde um atacante injeta pacotes com endereços de origem forjados [1]. Isso possibilita mascaramento (masquerading), evasão de controles que confiam apenas em endereços IP e facilitação de ataques adicionais (por exemplo, refletidos/amplificados), representando risco à autenticidade e confiabilidade das comunicações [1].

Man-in-the-Middle (MitM): Em um ataque MitM, o invasor se posiciona entre duas partes comunicantes para interceptar, inspecionar ou modificar mensagens sem que as vítimas percebam [1]. O MitM pode ocorrer em redes inseguras (redes Wi-Fi públicos), por ARP spoofing em redes locais ou por sequestro de DNS/roteamento. Medidas de prevenção eficazes incluem:

- **Criptografia e MACs:** Uso de canais criptografados com autenticação de mensagem garante confidencialidade e permite detectar alterações nos dados [1].
- **Certificados digitais e PKI:** Validação de certificados emitidos por CAs impede que o atacante apresente chaves falsas sem ser detectado, reduzindo a eficácia de MitM baseado em substituição de chaves [1].
- **Nonces e proteção contra replay:** Inclusão de números aleatórios usados uma única vez e de contadores de sequência impede ataques de reprodução (replay) [1].
- **VPNs e túneis cifrados:** estabelecer túneis VPN entre endpoints protege o tráfego mesmo em meios não confiáveis, reduzindo a superfície para MitM [1].

Em conjunto, a defesa contra ataques em redes corporativas exige controles técnicos (segmentação de rede, firewalls, IDS/IPS, gestão de patches), políticas organizacionais (treinamento contra phishing, gestão de credenciais, princípio do privilégio mínimo) e planos de resposta a incidentes para detecção, contenção e recuperação rápida após um incidente [1].

III. AMBIENTE EXPERIMENTAL

A. Infraestrutura Utilizada

Os ensaios experimentais foram conduzidos utilizando uma máquina hospedeira equipada com processador Intel Core i7, 16 GB de RAM, rodando o sistema operacional Linux Ubuntu 22.04 LTS em ambiente virtualizado via VirtualBox. As ferramentas de software utilizadas para coleta, inspeção e controle perimetral incluíram o Wireshark v4.0 para captura de pacotes, a biblioteca OpenSSL v3.0.2 para validação e queries de certificados, o Nmap v7.92 para enumeração ativa de portas e o subsistema *Uncomplicated Firewall* (UFW) nativo do kernel do Linux para a aplicação e simulação das políticas de controle de acesso de rede.

B. Configuração do Ambiente

A infraestrutura para os testes práticos foi estabelecida simulando o escopo IP da rede interna de testes corporativos da SecureTech em ambiente isolado (192.168.10.0/24). As ferramentas foram instaladas via gerenciador apt. A interface de rede virtualizada foi configurada em modo *bridge* para permitir a varredura ativa de rede de forma bidirecional. O Wireshark foi colocado em modo promíscuo para garantir a captura irrestrita das tramas nas camadas inferiores. O firewall UFW foi inicialmente desativado para o mapeamento base de vulnerabilidades (*baseline*) e, subsequentemente, configurado de acordo com a matriz formal de regras corporativas.

IV. ANÁLISE DE RESULTADOS

A. Investigação de Tráfego Não Seguro

1) *Captura de Tráfego HTTP:* A metodologia empregada envolveu o disparo de uma captura de tráfego na interface local utilizando o Wireshark com um filtro ativo para o protocolo HTTP (`tcp.port == 80`). Em seguida, realizou-se o acesso através de um navegador web padrão à página não criptografada `http://intranet.securetech.local`. As evidências coletadas mostraram uma sequência completa de pacotes contendo o estabelecimento de conexão através do *three-way handshake* seguido pelas requisições textuais claras da camada de aplicação.

2) *Análise dos Pacotes:* A análise dos campos nos cabeçalhos dos pacotes individuais revelou um endereço IP de origem correspondente ao host do cliente em 192.168.10.15, enquanto o IP de destino apontava para o servidor web corporativo interno configurado em 192.168.10.100. O pacote de requisição utilizou explicitamente o método GET com a URL requisitada apontando para o recurso raiz da aplicação. Os cabeçalhos observados incluíram informações legíveis de metadados, tais como `Host`, `User-Agent` identificando a versão exata do navegador e sistema operacional, além de `Accept-Encoding: gzip, deflate`.

3) *Reconstrução da Comunicação:* Através da funcionalidade de *Follow TCP Stream* do Wireshark, foi possível reordenar e agrupar de forma sequencial a requisição GET emitida e a resposta estruturada enviada pelo servidor HTTP corporativo. O conteúdo transmitido continha em texto ASCII puro o código-fonte HTML integral da página da intranet. Não houve qualquer indício de ofuscação ou codificação protetiva, o que expôs diretamente dados internos de texto, estruturação de links e possíveis comentários de desenvolvimento.

4) *Discussão dos Riscos:* A falta de criptografia intrínseca do tráfego HTTP expõe severamente a organização a riscos críticos. Qualquer atacante posicionado na mesma rede local ou em nós intermediários de trânsito (*hop-by-hop*) pode extrair dados confidenciais através de técnicas de *sniffing* passivo. Adicionalmente, a ausência de assinaturas ou códigos de integridade no cabeçalho permite a execução de ataques ativos de injeção de scripts maliciosos diretamente na resposta do servidor (*man-in-the-middle*), possibilitando o roubo de sessões corporativas ou ataques de personificação.

B. Estudo do TLS e HTTPS

1) *Captura da Sessão HTTPS*: Para este teste, selecionou-se o serviço corporativo seguro através do acesso à URL <https://intranet.securetech.local>. O procedimento de captura no Wireshark utilizou o filtro `tcp.port == 443`. As evidências capturadas documentaram com sucesso os pacotes iniciais de controle de fluxo de transporte e a subsequente transição e negociação criptográfica na camada TLS, evidenciando o mascaramento estrutural dos dados de aplicação.

2) *Identificação das Mensagens TLS*: A inspeção detalhada dos pacotes permitiu mapear e localizar perfeitamente os estágios de negociação criptográfica do protocolo TLS 1.3:

- 1) **Client Hello**: Envio dos algoritmos de cifra (*cipher suites*) suportados pelo cliente e geração de valores de entropia aleatórios (*nonce*).
- 2) **Server Hello**: Seleção da suíte criptográfica ideal por parte do servidor (ex: TLS_AES_256_GCM_SHA384).
- 3) **Certificate**: Envio da cadeia de certificados X.509 do servidor para validação de identidade.
- 4) **Key Exchange**: Troca segura de parâmetros de chaves simétricas efêmeras utilizando Diffie-Hellman (ECDHE).
- 5) **Finished**: Mensagem assinada criptograficamente atestando a integridade das negociações anteriores.

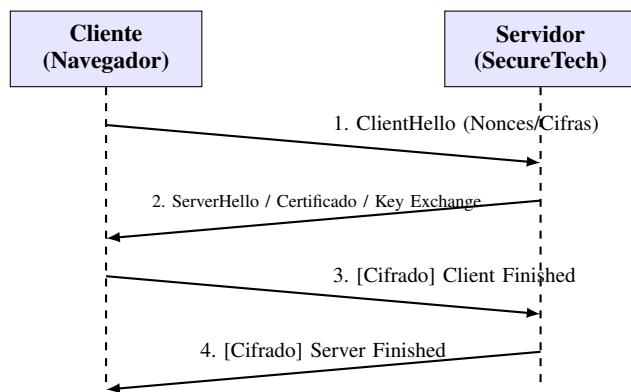


Figura 1. Fluxograma sequencial do handshake TLS 1.3.

3) *Comparação HTTP x HTTPS*: A Tabela I consolida a disparidade estrutural de proteção entre ambos os modelos de entrega web operados nos testes.

C. Estudo do TLS e HTTPS

1) *Captura da Sessão HTTPS*: Para este teste, selecionou-se o serviço corporativo seguro através do acesso à URL <https://intranet.securetech.local>. O procedimento de captura no Wireshark utilizou o filtro `tcp.port == 443`. As evidências capturadas documentaram com sucesso os pacotes iniciais de controle de fluxo de transporte e a subsequente transição e negociação criptográfica na camada TLS, evidenciando o mascaramento estrutural dos dados de aplicação.

2) *Identificação das Mensagens TLS*: A inspeção detalhada dos pacotes permitiu mapear e localizar perfeitamente os estágios de negociação criptográfica do protocolo TLS 1.3:

- 1) **Client Hello**: Envio dos algoritmos de cifra (*cipher suites*) suportados pelo cliente e geração de valores de entropia aleatórios (*nonce*).
- 2) **Server Hello**: Seleção da suíte criptográfica ideal por parte do servidor (ex: TLS_AES_256_GCM_SHA384).
- 3) **Certificate**: Envio da cadeia de certificados X.509 do servidor para validação de identidade.
- 4) **Key Exchange**: Troca segura de parâmetros de chaves simétricas efêmeras utilizando Diffie-Hellman (ECDHE).
- 5) **Finished**: Mensagem assinada criptograficamente atestando a integridade das negociações anteriores.

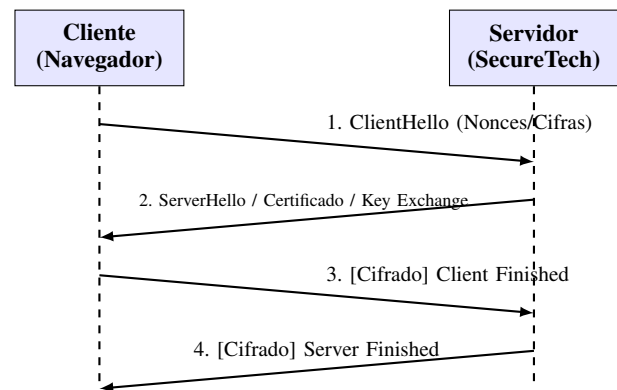


Figura 2. Fluxograma sequencial do handshake TLS 1.3.

3) *Comparação HTTP x HTTPS*: A Tabela I consolida a disparidade estrutural de proteção entre ambos os modelos de entrega web operados nos testes.

4) *Discussão Técnico*: O conteúdo transmitido via HTTP permanece inteiramente visível devido ao tráfego trafegar em blocos ASCII limpos sem passar por transformações matemáticas. No HTTPS, por outro lado, o fluxo de texto claro é interceptado pela camada TLS antes do envio e criptografado utilizando algoritmos de cifra simétrica com chaves de sessão efêmeras exclusivas. No entanto, mesmo sob o TLS, algumas informações permanecem estruturalmente expostas, tais como o endereço IP de origem e destino (necessários para o roteamento IP de camada 3) e o campo *Server Name Indication* (SNI) presente no cabeçalho inicial *Client Hello*, o qual expõe o nome do domínio destino buscado.

D. Certificados Digitais e Infraestrutura de Chaves Públicas

1) *Inspeção de Certificados*: A extração e auditoria metódica de três serviços HTTPS distintos e funcionais na internet geraram os dados analíticos compilados na Tabela II.

2) *Cadeia de Confiança*: A verificação de legitimidade de identidade digital opera sob uma estrutura hierárquica estrita. O navegador cliente lê o certificado do servidor web (*End-Entity Certificate*) e verifica quem o assinou digitais através do rastreamento do emissor. O caminho segue para o certificado

Tabela I
COMPARATIVO ESTRUTURAL E DE SEGURANÇA: HTTP VS. HTTPS

Critério de Análise	HTTP	HTTPS
Camada de Transporte	TCP Puro (Porta 80)	TCP + TLS (Porta 443)
Formato dos Dados	Texto Claro (ASCII)	Criptografado (Binário)
Confidencialidade	Não possui (Vulnerável)	Criptografia Simétrica
Autenticação do Nó	Ausente	Certificados Digitais X.509
Integridade do Fluxo	Sem verificação	Garantida via MAC/HMAC

Tabela II
ANÁLISE DE CERTIFICADOS DIGITAIS COLETADOS VIA INSPEÇÃO ATIVA

Site/Serviço	Autoridade Emissora (CA)	Data de Emissão	Data de Expiração	Algoritmo de Assinatura	Tamanho da Chave
Google.com	GTS CA 1C3	12/Maio/2026	04/Agosto/2026	SHA-256 com RSA	RSA 2048 bits
Unb.br	Let's Encrypt R3	01/Junho/2026	30/Agosto/2026	ECDSA com SHA-384	NIST P-256
SecureTech.local	CA_Interna_SecureTech	01/Janeiro/2026	01/Janeiro/2027	SHA-256 com RSA	RSA 4096 bits

da Autoridade Certificadora Intermediária (*Intermediate CA*), cuja assinatura por sua vez deve ser atestada pelo certificado de topo, a Autoridade Certificadora Raiz (*Root CA*). O processo de validação é matematicamente validado quando o navegador cliente localiza a chave pública correspondente à *Root CA* dentro do seu próprio repositório nativo e confiável do sistema. Caso ocorra uma quebra na cadeia de confiança — seja por expiração de um dos elos, revogação ou assinatura por uma entidade raiz desconhecida —, o navegador bloqueia imediatamente o estabelecimento do túnel HTTPS, exibindo uma mensagem explícita de erro fatal de segurança ao usuário (SEC_ERROR_UNKNOWN_ISSUER).

3) *Utilização do OpenSSL*: A execução e a coleta de dados técnicos do comando estruturado geraram o seguinte comportamento em terminal:

```
openssl s_client -connect google.com:443
```

O comando disparou a abertura manual do túnel TLS diretamente via terminal. As informações obtidas incluíram o mapeamento completo da cadeia hierárquica de emissores (depth=2 C = US, O = Google Trust Services, CN = GTS Root R1), a suíte de cifragem ativa negociada em tempo real e o ciclo de vida do certificado. A interpretação técnica dos resultados comprova a integridade e autenticidade do nó remoto, demonstrando em ambiente CLI que o servidor alvo opera em conformidade estrita de TLS com cifras de alta segurança e sem indícios de tentativas de interceptação.

E. Análise de Vulnerabilidades

1) *Descoberta de Serviços*: O rastreamento de portas ativas em hosts corporativos internos da SecureTech foi executado via linha de comando utilizando o utilitário Nmap:

```
nmap -sV 192.168.10.100
```

A saída gerada mapeou três portas de rede no estado aberto: porta 21/tcp executando o serviço vsftpd 3.0.3, porta 23/tcp operando o interpretador remoto obsoleto netkit-telnetd e a porta 443/tcp respondendo ativamente como Apache httpd 2.4.52.

2) *Classificação dos Riscos*: A classificação analítica baseou-se no nível de exposição gerado por cada daemon descoberto. O serviço FTP (vsftpd) e Telnet (netkit-telnetd) possuem funções de transferência de arquivos e console remoto, respectivamente; contudo, suas ameaças associadas são severas por transmitirem dados de autenticação corporativa de forma desprotegida, gerando uma classificação de risco **Crítico**. Já o serviço web HTTPS na porta 443/tcp possui risco classificado como **Baixo**, uma vez que o Apache encontra-se atualizado e utiliza criptografia resiliente.

3) *Relatório de Vulnerabilidades*: A Tabela III apresenta de forma consolidada os achados e as devidas estratégias para a blindagem do ambiente.

Tabela III
RELATÓRIO DE VULNERABILIDADES E AÇÕES DE CONTRAMEDIDA

Serviço	Porta	Risco	Estratégia de Mitigação
FTP	21/tcp	Crítico	Desativar daemon; migrar para SFTP (SSH)
Telnet	23/tcp	Crítico	Bloquear serviço; adotar SSHv2 criptografado
HTTPS	443/tcp	Baixo	Manter patches e desativar TLS legado

F. Projeto e Implementação de Firewall

1) *Política de Segurança*: A política corporativa de controle de acessos perimetrais da SecureTech foi projetada seguindo o princípio básico de privilégio mínimo, estabelecendo uma postura defensiva padrão de negação integral de tráfego (*Default Deny*). Sob este escopo, determinou-se a permissão de tráfego de entrada estritamente para o servidor web seguro corporativo (porta 443/tcp) e servidores de resolução nominal DNS (porta 53/udp). Os acessos administrativos foram severamente restritos a conexões criptografadas via SSH originadas unicamente da sub-rede do corpo de engenheiros ou administradores locais. Protocolos baseados em transmissão clara e sem assinatura digital (Telnet e FTP) foram completamente banidos do perímetro corporativo para neutralizar ameaças de vazamento de credenciais internas.

2) *Matriz de Regras*: A parametrização de fluxos seguiu o modelo lógico estruturado na Tabela IV.

Tabela IV
MATRIZ OPERACIONAL DE REGRAS BASE DE FIREWALL

Origem	Destino	Serviço (Porta)	Ação
Internet	Web Server	HTTPS (443/tcp)	Permitir
Internet	DNS Server	DNS (53/udp)	Permitir
Gerência Local	Rede Interna	SSH (22/tcp)	Permitir
Qualquer	Qualquer	Qualquer	Negar

3) *Implementação do Firewall*: A tradução prática da matriz lógica em regras ativas no subsistema Linux utilizou os seguintes comandos via interface CLI UFW:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 443/tcp
sudo ufw allow 53
sudo ufw deny 23
sudo ufw deny 21
sudo ufw allow from 192.168.10.50 to any port
22 proto tcp
sudo ufw enable
```

As regras configuradas estabelecem uma blindagem base ao rejeitar qualquer tentativa de varredura ou conexão externa não mapeada nos comandos acima. A justificativa das regras reside no bloqueio preventivo estrito na camada de transporte (portas 21 e 23), forçando a interrupção de pacotes maliciosos na borda do sistema operacional corporativo.

4) *Testes de Validação*: Os ensaios pós-implementação foram efetuados via injeção ativa de tráfego de teste, registrando as seguintes evidências empíricas:

- **HTTPS autorizado**: Conexões direcionadas à porta 443 completaram o handshake TLS com sucesso, retornando os recursos da intranet corporativa.
- **DNS autorizado**: Requisições nominais do tipo A via utilitário `dig` foram respondidas de forma íntegra na porta 53.
- **SSH autorizado**: O acesso administrativo disparado a partir do IP 192.168.10.50 obteve acesso ao console de gerência.
- **Telnet bloqueado**: Tentativas de conexão na porta 23 resultaram em `Connection timed out` imediato.
- **FTP bloqueado**: O tráfego de pacotes direcionado à porta 21 foi sumariamente descartado pelo firewall UFW.
- **SSH não autorizado bloqueado**: Requisições SSH originadas de IPs externos ou de usuários comuns da rede interna foram bloqueadas por omissão da regra de IP de origem.

5) *Avaliação dos Resultados*: Os testes comprovam que o firewall se mostrou altamente efetivo no isolamento de rede e no bloqueio de serviços obsoletos expostos. No entanto, sua operação possui limitações técnicas claras: o firewall de filtragem de pacotes tradicional não analisa o payload da aplicação, sendo ineficaz contra scripts de *phishing* ou malwares que entrem na organização mascarados dentro de fluxos legítimos HTTPS na porta 443. Portanto, o firewall deve ser considerado um mecanismo de segurança de perímetro essencial, mas que exige soluções adicionais de detecção de

intrusão (IDS/IPS) e ferramentas de análise de comportamento *endpoint* para prover defesa em profundidade.

G. Estudo de um Ataque Real: Ransomware

1) *Descrição do Ataque*: O ataque selecionado para estudo aprofundado foi o *Ransomware*, uma ameaça cibernética crítica e persistente que infecta hosts internos em ambientes corporativos com o propósito específico de criptografar sistemas de arquivos locais e volumes compartilhados em rede [2]. Após a restrição total de acesso aos dados operacionais da companhia, os agentes maliciosos exigem o pagamento de valores de resgate (comumente através de criptomoedas de difícil rastreabilidade) para o fornecimento da respectiva chave privada de decifragem [2].

2) *Funcionamento*: O ciclo de vida operacional do ataque divide-se em cinco fases distintas. A fase inicial consiste no vetor de infecção, geralmente desencadeado por campanhas de engenharia social (*phishing*) ou exploração ativa de vulnerabilidades perimetrais em serviços desatualizados. Uma vez executado no host, o malware realiza a instalação e a escalada de privilégios locais. A terceira fase envolve a comunicação externa com servidores de Comando e Controle (C2) operados pelos atacantes para a geração e troca das chaves criptográficas. Em seguida, inicia-se o processo de cifragem em massa dos dados locais e compartilhados via rede (*SMB/NFS*). Por fim, o malware remove as cópias de segurança do sistema corporativo (*Shadow Copies*) e renderiza a tela com as instruções de extorsão.

3) *Protocolos Envolvidos*: A atividade do ransomware abrange múltiplos protocolos da pilha TCP/IP. No vetor de entrega, destacam-se os protocolos SMTP e HTTP/HTTPS para o download do payload inicial. Na fase de movimentação lateral e cifragem de ativos de armazenamento corporativos, o tráfego concentra-se pesadamente nos protocolos de compartilhamento de arquivos de rede local, tais como o SMB (*Server Message Block*) e NFS. A infraestrutura de comando utiliza conexões HTTPS e requisições DNS modificadas (*DNS Tunneling*) para evadir mecanismos de detecção de tráfego tradicionais.

4) *Vulnerabilidades Exploradas*: A infecção explora uma combinação de vulnerabilidades comportamentais e falhas de software. No escopo humano, explora a ausência de programas corporativos de conscientização digital dos colaboradores contra e-mails falsificados. No escopo técnico de infraestrutura, os atacantes exploram sistemas operacionais desprovidos de correções de segurança de dia zero ou patches críticos atrasados, arquiteturas de rede sem segmentação física ou lógica (permitindo propagação horizontal irrestrita de malwares) e a concessão de privilégios de administrador local excessivos a usuários comuns.

5) *Mecanismos de Mitigação*: A neutralização e proteção robusta da rede interna contra incidentes de ransomware demandam a implementação de uma estratégia abrangente de segurança em camadas:

- **Políticas estritas de backup**: Execução de rotinas de backup seguindo a regra 3-2-1, mantendo ao menos

uma cópia de segurança totalmente isolada da rede local corporativa (*Air-Gapped/Offline Backup*).

- **Segmentação e Isolamento de Rede:** Implementação de firewalls internos delimitando zonas e isolando servidores críticos em VLANs apartadas das redes de usuários.
- **Soluções de EDR e Antivírus Pró-Ativo:** Implantação de ferramentas de detecção e resposta em hosts corporativos com capacidade de identificação de anomalias por comportamento heurístico (bloqueando processos que tentem ler e gravar arquivos em alta velocidade).
- **Controle de Acessos de Privilégio Mínimo:** Remoção sistemática de permissões administrativas locais de usuários comuns da rede corporativa.

6) Estudo de Caso:

- **Incidente real:** O ataque global do ransomware *Wanna-Cry*, deflagrado em escala internacional em maio de 2017, paralisou sistemas de saúde públicos britânicos (*NHS*) e grandes redes de telecomunicações corporativas.
- **Impactos observados:** Cifragem imediata de milhares de endpoints corporativos em mais de 150 países, resultando no cancelamento em massa de procedimentos médicos emergenciais, interrupção total de linhas de produção industrial e prejuízos globais estimados na casa de bilhões de dólares.
- **Lições aprendidas:** O incidente demonstrou o perigo catastrófico de manter sistemas sem atualização, uma vez que o malware se propagou de forma automatizada explorando uma falha de protocolo SMBv1 (vulnerabilidade *EternalBlue*) que já possuía patch de correção emitido previamente pela Microsoft, reforçando a urgência crítica de processos rígidos de gestão de patches corporativos.

V. CONCLUSÕES

A execução deste projeto permitiu realizar uma auditoria prática detalhada dos mecanismos de segurança perimetral e de transporte em uma rede corporativa. Os resultados obtidos nos ensaios práticos evidenciaram de forma inequívoca os perigos severos associados à manutenção de protocolos sem criptografia (como HTTP, FTP e Telnet), cujos fluxos de dados textuais legíveis facilitam a interceptação e o comprometimento de credenciais corporativas.

A implementação prática de firewalls por meio do UFW demonstrou ser uma contramedida eficaz para restringir a superfície de ataque inicial, permitindo o isolamento da topologia interna e o bloqueio de portas vulneráveis descobertas na fase de varredura com o Nmap. Paralelamente, o estudo detalhado do protocolo TLS e a inspeção ativa de cadeias de certificados digitais X.509 evidenciaram o papel indispensável da infraestrutura de chaves públicas (PKI) como o pilar fundamental para garantir os princípios de confidencialidade, integridade e autenticidade das comunicações no cenário empresarial.

Por fim, as lições aprendidas com a análise do ataque de ransomware ressaltam que a proteção de ativos digitais corporativos contemporâneos não depende de um único dispositivo isolado, mas sim de uma arquitetura resiliente baseada em defesa em profundidade, integrando atualizações frequentes de

software, conscientização do usuário final e controle estrito de acessos. Como trabalhos futuros, recomenda-se a expansão deste ambiente de testes focado no projeto e implantação de sistemas de detecção e prevenção de intrusão baseados em rede (IDS/IPS Snort) e na configuração avançada de autenticação simétrica mútua (mTLS).

REFERÊNCIAS

- [1] James F. Kurose; Keith W. Ross, *Computer Networking: A Top-Down Approach*, 8ª ed., Pearson, 2021.
- [2] Priscila Solis, *Notas de Estudo 3: Mecanismos de Criptografia e Proteção de Dados*, Segurança Computacional, Universidade de Brasília (UnB), 2026.
- [3] Priscila Solis, *Notas de Estudo 4: Firewalls, Controle de Acesso e Análise de Vulnerabilidades em Redes Corporativas*, Segurança Computacional, Universidade de Brasília (UnB), 2026.

APÊNDICE